基于Anaconda安装Python开发环境

这个文档主要帮助不熟悉Python开发的朋友快速构建Python语言开发环境

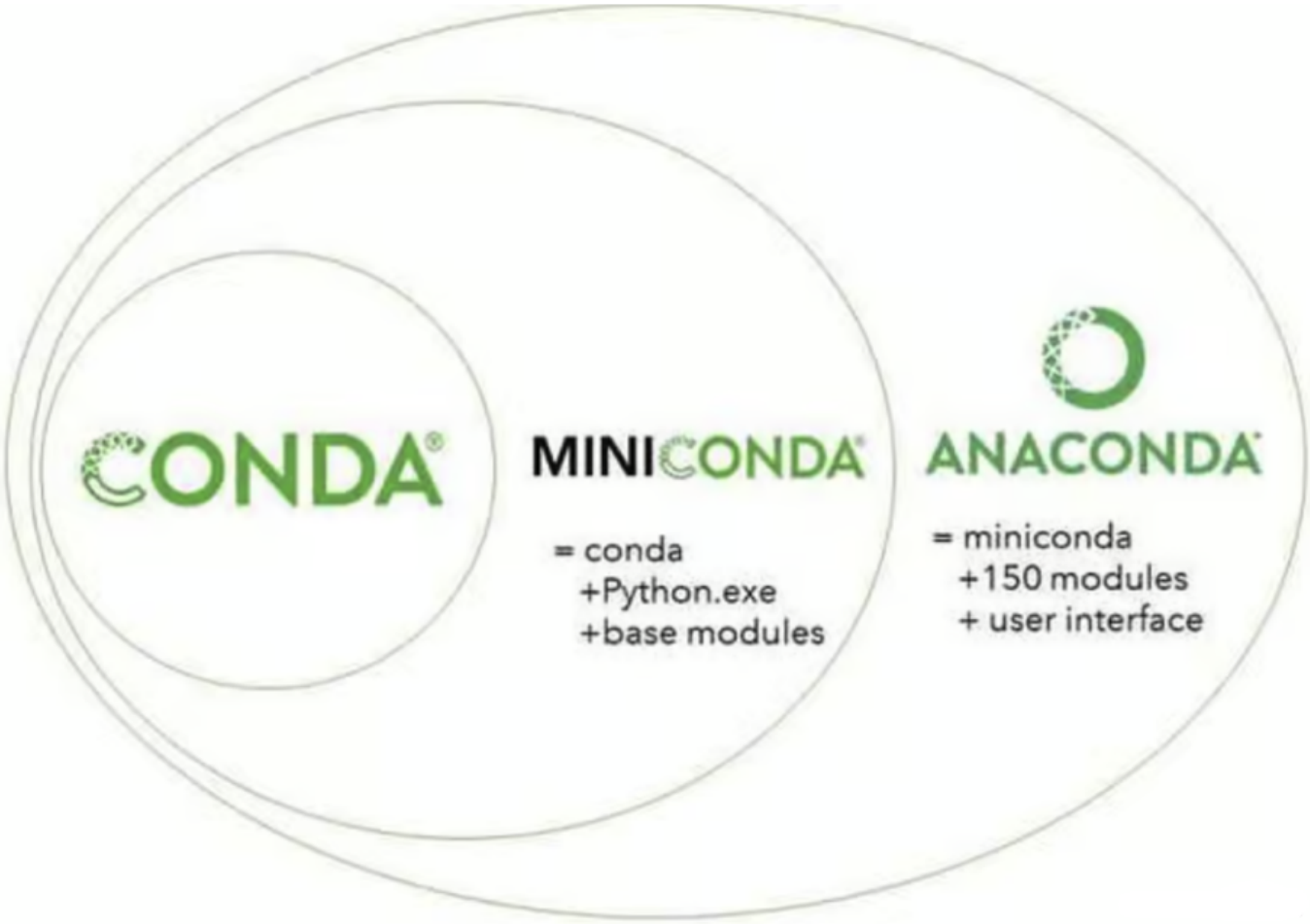
Python与Anaconda

Python是一种面向对象的高级脚本语言。相比于C、Java等高级语言，Python语言非常简洁。相同的逻辑，使用Python开发，通常可以减少很多代码量。Python的语法设计非常易读，阅读编写良好的Python代码几乎没有语法门槛。Python是解释型语言，开发过程不需要编译成二进制代码，开发过程非常快。多年发展过程中，基于Python积累了非常多经典的扩展库，提供了方方面面的功能，这使得应用开发通常也非常简单。Python的这些良好易用的特性，使得Pyhon成为了科研界的首选。

在Python的世界里，有非常多的扩展库，这些扩展库在应用开发时扮演了非常重要的角色。这时，就需要一个工具来管理这些各种各样的扩展库。这个工具就是Conda。基于Conda可以快速构建不同版本的Python运行环境，并且可以快速管理各种第三方库。此外，Conda还可以跨平台使用，使开发者在不同的操作系统上都能够轻轻使用和管理这些扩展库。

接下来我们使用的Anaconda是一个开源的Python版本。他包含了conda，Python以及上百个常用的科学研究包和依赖性。如pandas，numpy，matplotlib等，这些都是进行数据分析、机器学习等任务的重要扩展包。

> 另外还有一个轻量级的Anacoda版本，就是Miniconda，他只包含Coda和Ptyhon，但没有包含Anaconda中捆绑的科学计算和数据分析的包。因此，Miniconda安装包更小，可以节省空间。但是在安装后，就需要自行选择安装哪些包。



快速使用Anaconda

Anaconda官方下载地址：https://www.anaconda.com/download
Anaconda国内下载地址：https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/

下载对应版本后，双击安装即可。安装成功后，可以使用conda --version指令查看版本。版本展示成功即表示服务安装成功。

常用conda命令

创建Python环境

创建一个Python环境：

```
conda create -n langchain_env python=3.10
```

上面指令会创建一个名为langchian_env的python3.10环境。执行命令后，需要输入y，确认安装一些默认依赖。

```
The following packages will be downloaded:

    package                    |            build
    ---------------------------|-----------------
    ca-certificates-2024.9.24  |       hca03da5_0         131 KB
    libffi-3.4.4               |       hca03da5_1         120 KB
    openssl-3.0.15             |       h80987f9_0         4.3 MB
    pip-24.2                   |   py38hca03da5_0         2.2 MB
    python-3.8.20              |       hb885b13_0        12.5 MB
    setuptools-75.1.0          |   py38hca03da5_0         1.6 MB
    sqlite-3.45.3              |       h80987f9_0         1.2 MB
    tk-8.6.14                  |       h6ba3021_0         3.3 MB
    wheel-0.44.0               |   py38hca03da5_0         110 KB
    xz-5.4.6                   |       h80987f9_1         371 KB
    zlib-1.2.13                |       h18a0788_1          91 KB
    ------------------------------------------------------------
                                           Total:        26.0 MB

The following NEW packages will be INSTALLED:

  ca-certificates    pkgs/main/osx-arm64::ca-certificates-2024.9.24-hca03da5_0
  libcxx             pkgs/main/osx-arm64::libcxx-14.0.6-h848a8c0_0
  libffi             pkgs/main/osx-arm64::libffi-3.4.4-hca03da5_1
  ncurses            pkgs/main/osx-arm64::ncurses-6.4-h313beb8_0
  openssl            pkgs/main/osx-arm64::openssl-3.0.15-h80987f9_0
  pip                pkgs/main/osx-arm64::pip-24.2-py38hca03da5_0
  python             pkgs/main/osx-arm64::python-3.8.20-hb885b13_0
  readline           pkgs/main/osx-arm64::readline-8.2-h1a28f6b_0
  setuptools         pkgs/main/osx-arm64::setuptools-75.1.0-py38hca03da5_0
  sqlite             pkgs/main/osx-arm64::sqlite-3.45.3-h80987f9_0
  tk                 pkgs/main/osx-arm64::tk-8.6.14-h6ba3021_0
  wheel              pkgs/main/osx-arm64::wheel-0.44.0-py38hca03da5_0
  xz                 pkgs/main/osx-arm64::xz-5.4.6-h80987f9_1
  zlib               pkgs/main/osx-arm64::zlib-1.2.13-h18a0788_1


Proceed ([y]/n)? y
```

输入y 确认

```
Downloading and Extracting Packages

Preparing transaction: done
Verifying transaction: done
Executing transaction: done
#
# To activate this environment, use
#
#     $ conda activate testenv
```

激活环境

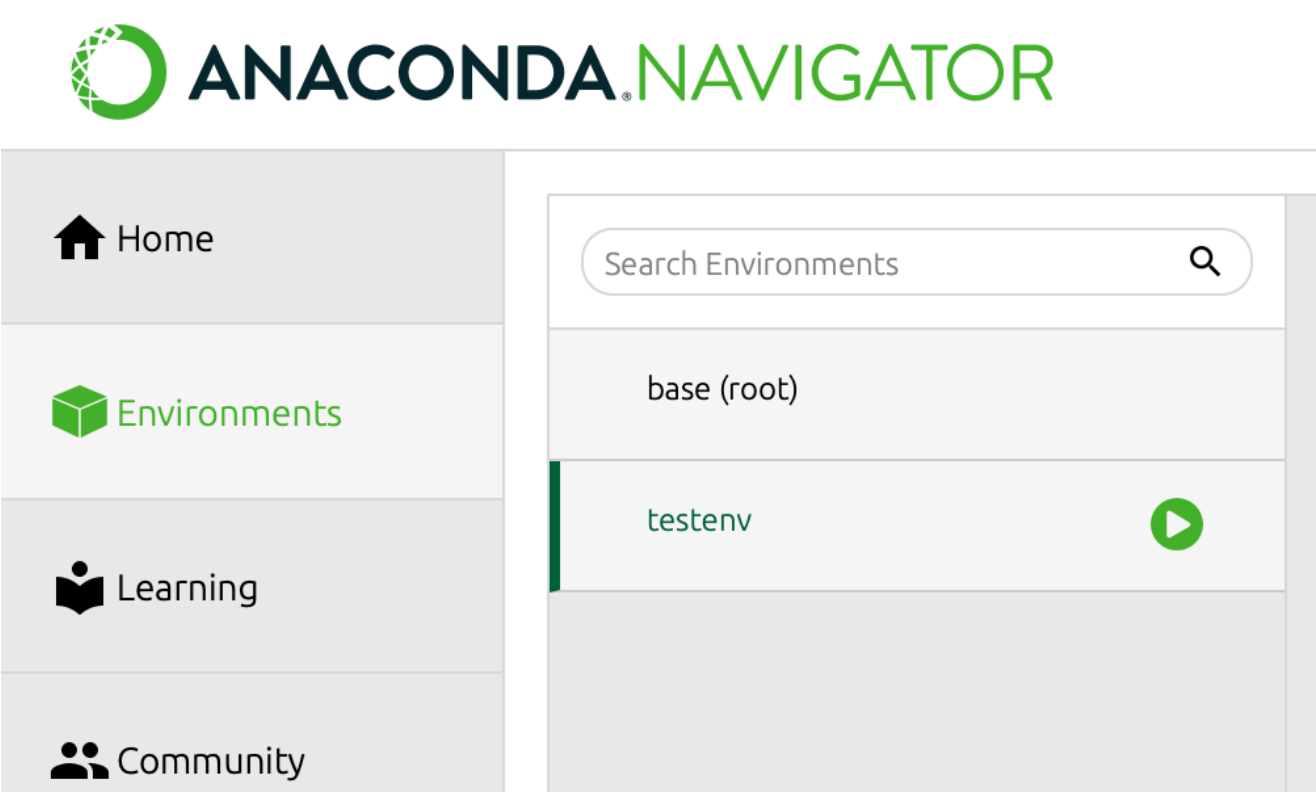
接下来，执行下面的指令就可以激活刚刚创建的环境：

```
conda activate langchain_env
```

激活这个环境后，之前安装的依赖项，就会在这个环境中生效。
接下来，可以通过一系列指令管理环境以及维护环境内的依赖

```
conda list #查看当前环境下的依赖
conda install xxx #安装依赖
# 通常会用pip install xxx 安装指令
conda remove xxx #删除依赖
conda update xxx #更新依赖
conda deactivate #退出环境
```

> 安装依赖时，使用conda安装会去分析依赖关系并确保兼容，安装速度通常比较慢。pip 安装通常只下载和安装对应的包以及依赖项，安装速度会比较快。
> 另外，在Anaconda中，也提供了图形化的管理工具Navigator，通过这个工具可以更方便的管理环境以及依赖。



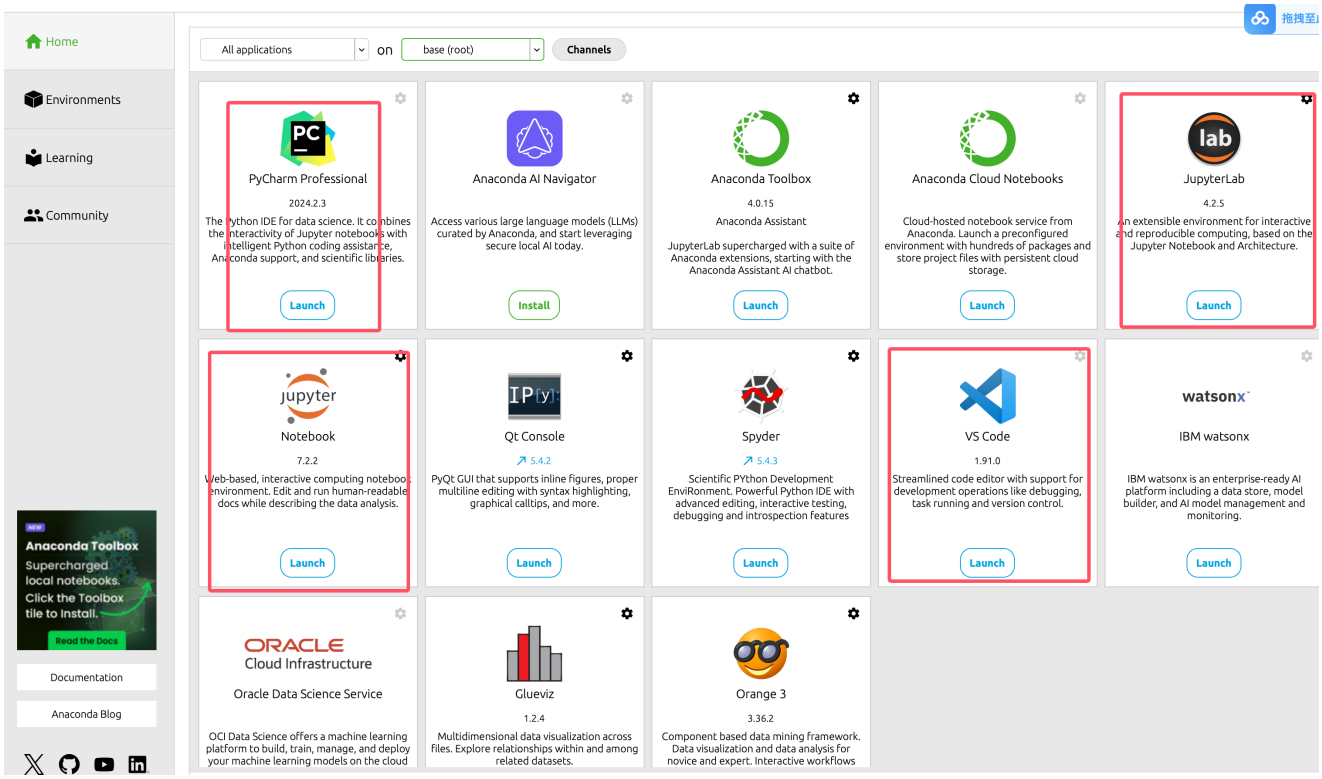
常用的Python开发工具

Python的运行环境是比较简单的。在命令行中直接敲python就可以进入交互式的Python环境。

```
(testenv) roykingw@roykingwdeMacBook-Pro ~ % python
Python 3.8.20 (default, Oct  3 2024, 10:22:23)
[Clang 14.0.6 ] :: Anaconda, Inc. on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> a='hello '
>>> b='world'
>>> print(a+b)
hello world
>>>
```

这种方式，适合于快速测试一些代码。但是在实际的开发中，我们通常使用集成开发环境（IDE）。
在Anaconda的Navigator中，就提供了一些常用的IDE。如果没有安装，在这个界面会直接引导安装对应的

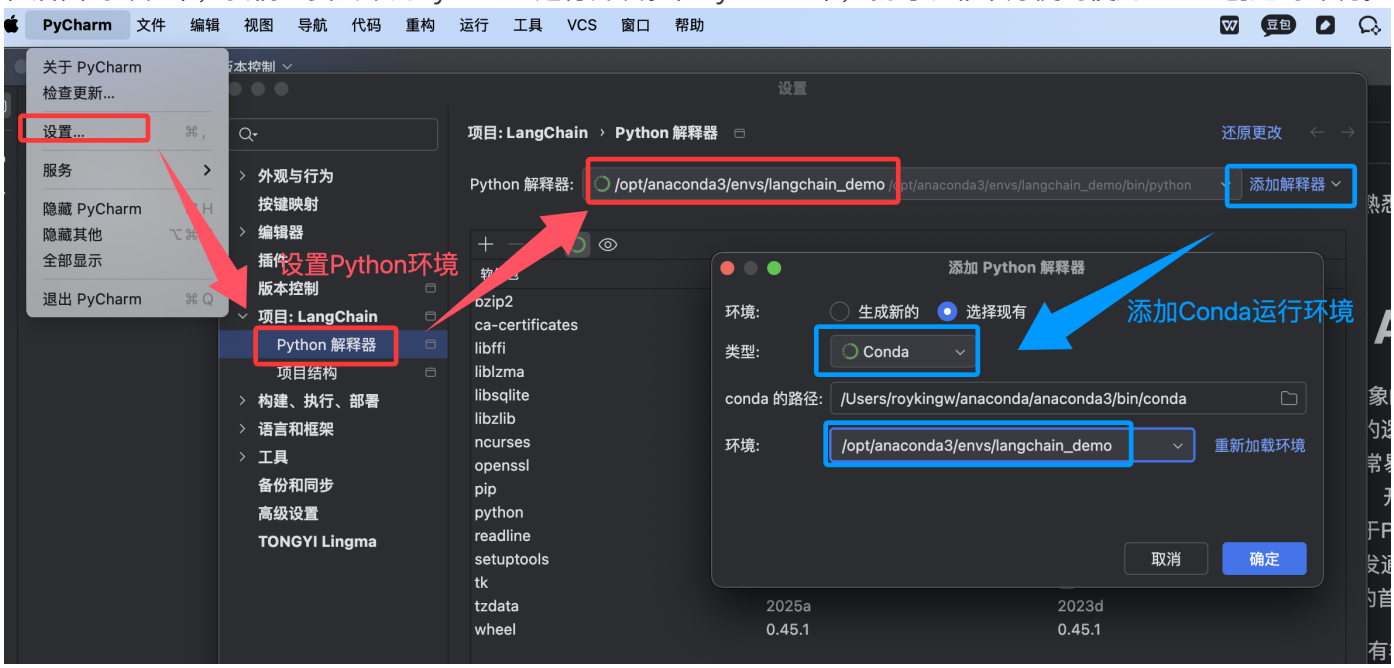
工具。



其中红色框框这几个工具，是开发过程中用得最多的几个工具。其中Pycharm和VsCode都是非常强大的集成式开发IDE。而Jupyter Notebook和JupyterLab则是一个非常方便的交互式开发工具，甚至在很多云服务器上，都会部署作为Python环境的运行工具。

PyCharm环境配置

在后面的课程中，我们主要会采用PyCharm进行开发。在Pycharm中，就可以非常方便的使用conda创建的环境。



选择环境后，就可以创建python项目了。Python的源文件通常有两种，其中.py是Python的源代码文件，里面就可以用来直接运行Python的代码。而.ipynb是Jupyter Notebook的源代码文件，文件中不光可以运行Python代码，还可以添加补充信息。

jupyter环境配置

Jupyter也是Python中一种常见的编程环境，他的源文件为.ipynd。运行Jupyter需要安装一些依赖。

```
# conda激活当前环境
pip install jupyter
```

接下来需要执行以下指令，可以在jupyter中注册内核环境，这样在jupyter中就可以指定环境进行编码了。

```
pip install ipykernel
python -m ipykernel install --user --name=langchain_env --display-
name="py310(langchain_env)"
```

新的环境安装完成后，就可以在pycharm或者jupyter nootbook的右上角切换对应的执行环境。比如上面创建的py310(langchain_env)环境。
接下来，开始愉快的基于Python的AI大模型之旅吧。